



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 02: C# parte 1

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

7 de Agosto, 2025

Motivación

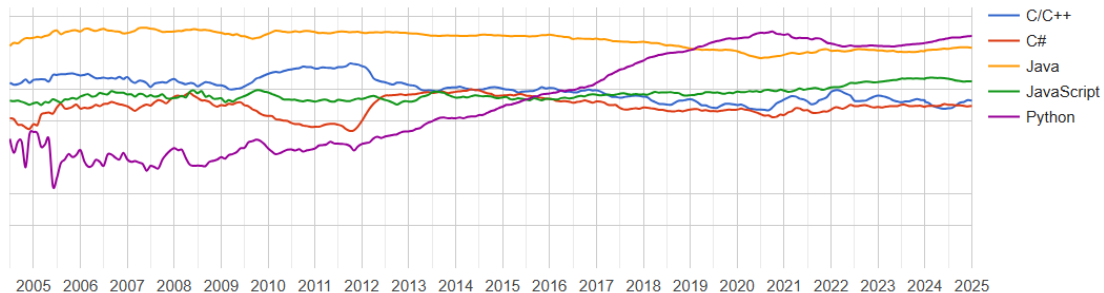
¿Por qué debería aprender a programar en C#?

¿Por qué debería aprender a programar en C#?



¿Por qué debería aprender a programar en C#?

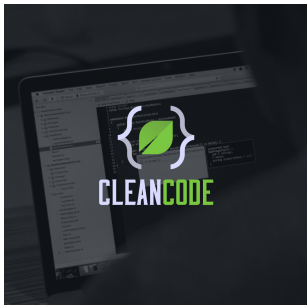
PYPL Popularity of Programming Language



¿Por qué debería aprender a programar en C#?



¿Por qué debería aprender a programar en C#?



Listing 3-5

Employee and Factory

```
public abstract class Employee {
    public abstract boolean isPayday();
    public abstract Money calculatePay();
    public abstract void deliverPay(Money pay);
}

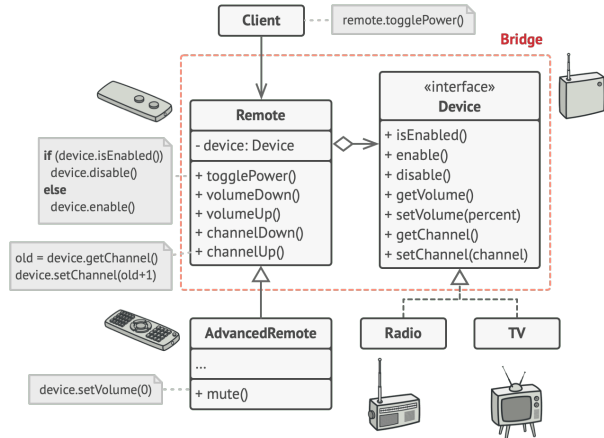
-----

public interface EmployeeFactory {
    public Employee makeEmployee(EmployeeRecord r) throws InvalidEmployeeType;
}

-----

public class EmployeeFactoryImpl implements EmployeeFactory {
    public Employee makeEmployee(EmployeeRecord r) throws InvalidEmployeeType {
        switch (r.type) {
            case COMMISSIONED:
                return new CommissionedEmployee(r);
            case HOURLY:
                return new HourlyEmployee(r);
            case SALARIED:
                return new SalariedEmployee(r);
            default:
                throw new InvalidEmployeeType(r.type);
        }
    }
}
```

¿Por qué debería aprender a programar en C#?



C# y Rider

Trabajaremos con .NET 8



<https://dotnet.microsoft.com/en-us/download>

Instalen Rider (puedes obtener una licencia de estudiante con tu correo UC).



<https://www.jetbrains.com/rider/>

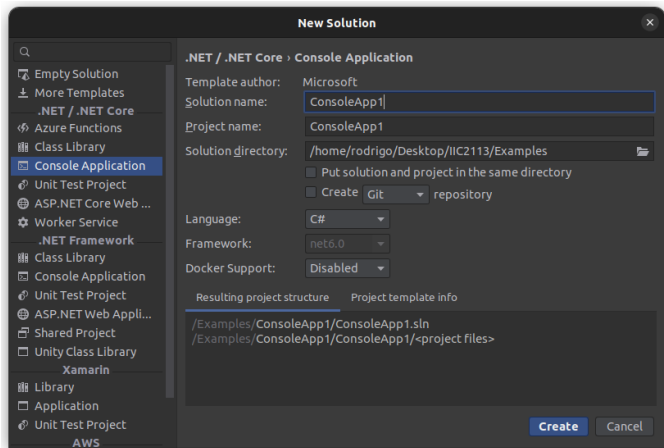
“Hello, World!”

“Hello, World!”

En Rider, crear un nuevo proyecto (`file/new...`)

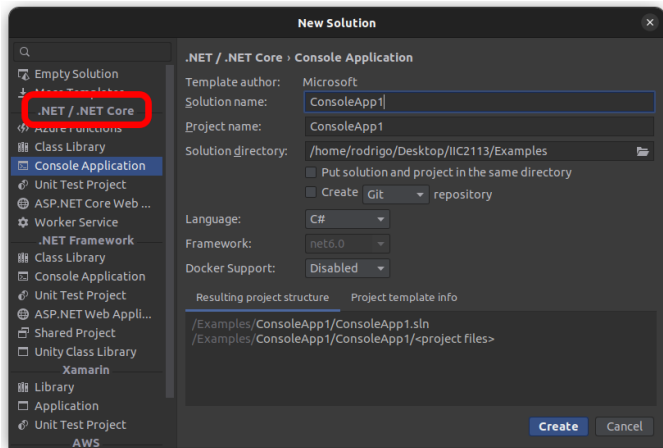
"Hello, World!"

En Rider, crear un nuevo proyecto (file/new...). Aparece lo siguiente:



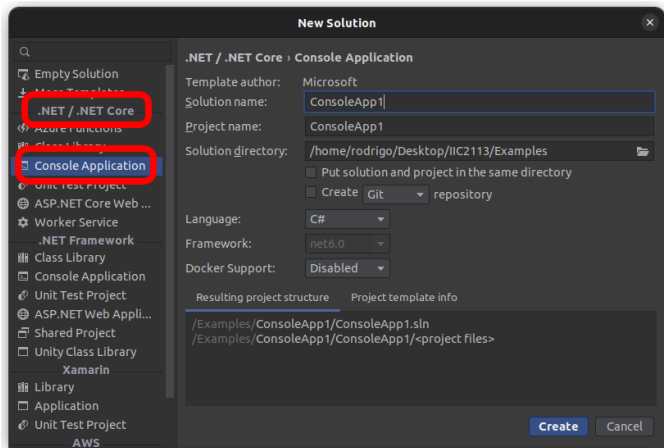
"Hello, World!"

En Rider, crear un nuevo proyecto (file/new...). Aparece lo siguiente:



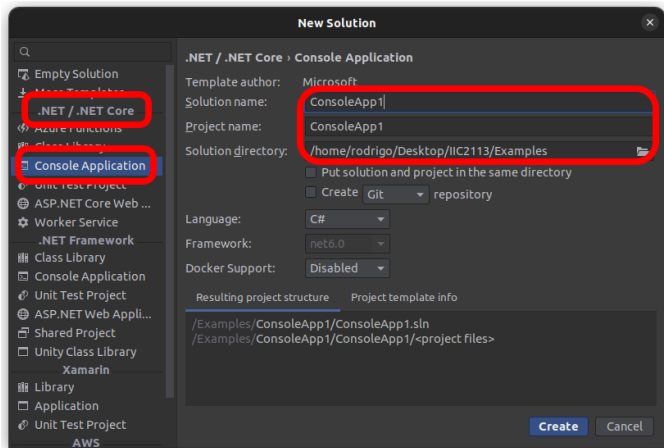
"Hello, World!"

En Rider, crear un nuevo proyecto (file/new...). Aparece lo siguiente:



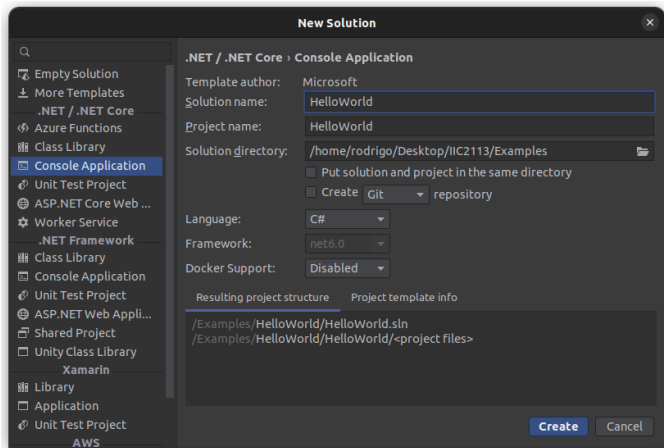
"Hello, World!"

En Rider, crear un nuevo proyecto (file/new...). Aparece lo siguiente:



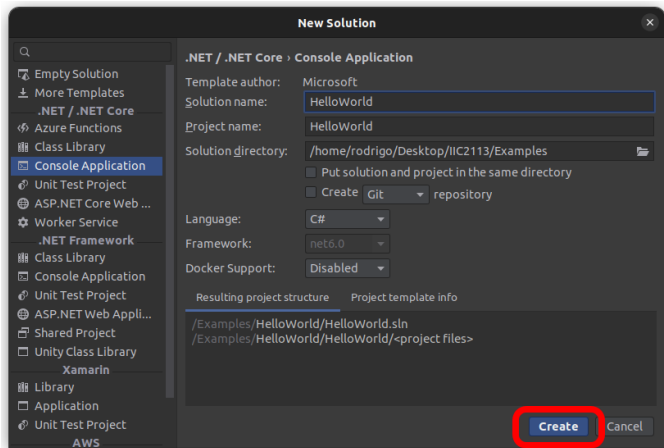
"Hello, World!"

En Rider, crear un nuevo proyecto (file/new...). Aparece lo siguiente:



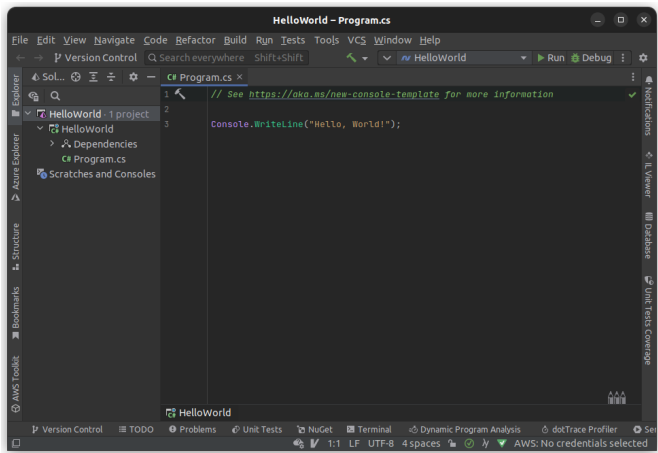
“Hello, World!”

En Rider, crear un nuevo proyecto (file/new...). Aparece lo siguiente:



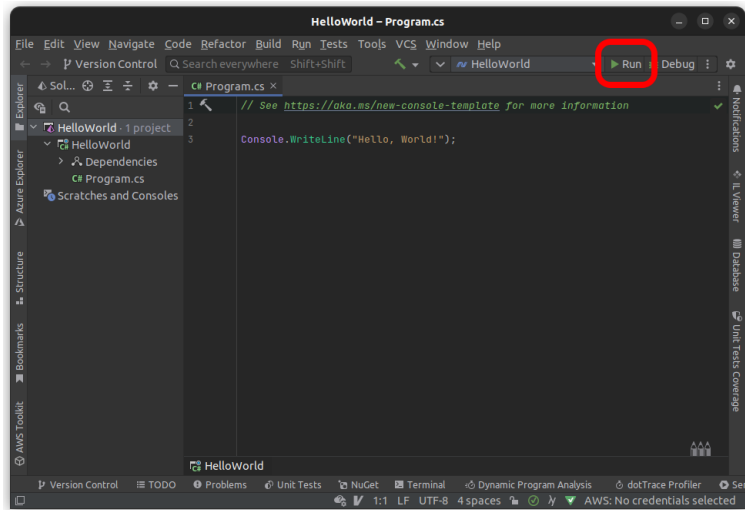
"Hello, World!"

En Rider, crear un nuevo proyecto (file/new...). Aparece lo siguiente:



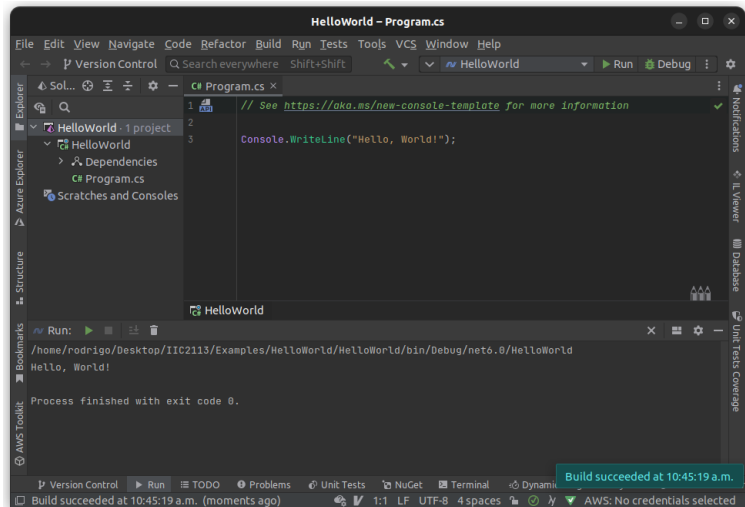
"Hello, World!"

En Rider, crear un nuevo proyecto (file/new...). Aparece lo siguiente:



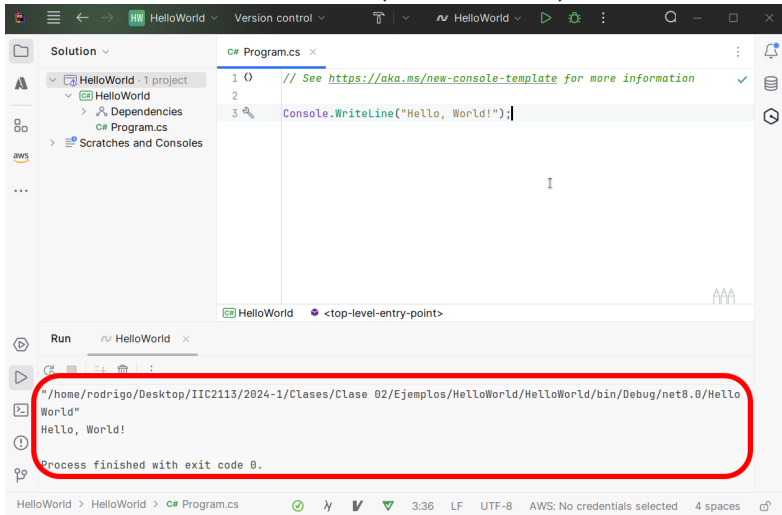
"Hello, World!"

En Rider, crear un nuevo proyecto (file/new...). Aparece lo siguiente:



"Hello, World!"

En Rider, crear un nuevo proyecto (file/new...). Aparece lo siguiente:



"Hello, World!"

Notar que al crear un nuevo proyecto de tipo consola se autogeneró el archivo Program.cs que muestra un "Hello, World!".

```
1 Console.WriteLine("Hello, World!");
```


"Hello, World!"

Notar que al crear un nuevo proyecto de tipo consola se autogeneró el archivo Program.cs que muestra un "Hello, World!".

```
1 Console.WriteLine("Hello, World!");
```

Por defecto, el main() de tu programa será el código que pongas en Program.cs. E.j., si cambiamos Program.cs a lo siguiente.

```
1 int i = 0;  
2 i++; // También se puede usar "i += 1;"  
3 Console.WriteLine(i);
```

se muestra en consola un 1.

"Hello, World!"

Notar que al crear un nuevo proyecto de tipo consola se autogeneró el archivo Program.cs que muestra un "Hello, World!".

```
1 Console.WriteLine("Hello, World!");
```

Por defecto, el main() de tu programa será el código que pongas en Program.cs. E.j., si cambiamos Program.cs a lo siguiente.

```
1 int i = 0;  
2 i++; // También se puede usar "i += 1;"  
3 Console.WriteLine(i);
```

se muestra en consola un 1.

Si el main está vacío, Rider se enojará

Elementos Básicos

Comentarios:

```
1 // Esto comenta una línea de código
2 /*
3     Así podemos comentar
4     varios líneas de código
5 */
```

Comentarios:

```
1 // Esto comenta una línea de código
2 /*
3     Así podemos comentar
4     varios líneas de código
5 */
```

Toda instrucción termina con “;” y los scopes se definen usando brackets:

```
1 int a = Convert.ToInt32(Console.ReadLine());
2 if (a > 0)
3 {
4     a = 2 * a;
5     Console.WriteLine(a);
6 }
```

A diferencia de Python, C# es un lenguaje *fuertemente tipado*. Es decir, toda variable debe ser *declarada* con su *tipo* antes de ser usada:

- `<tipo> <nombre variable>;`
- `<tipo> <nombre variable> = <valor>;`

Elementos Básicos

A diferencia de Python, C# es un lenguaje *fuertemente tipado*. Es decir, toda variable debe ser *declarada* con su *tipo* antes de ser usada:

- `<tipo> <nombre variable>;`
- `<tipo> <nombre variable> = <valor>;`

Ejemplo:

```
1 int a; // Declarar una variable
2 a = 5; // Asignar un valor a una variable
3 int b = 3; // Declarar y asignar a la vez
4 Console.WriteLine(a*b);
```

Elementos Básicos

A diferencia de Python, C# es un lenguaje *fuertemente tipado*. Es decir, toda variable debe ser *declarada* con su *tipo* antes de ser usada:

- `<tipo> <nombre variable>;`
- `<tipo> <nombre variable> = <valor>;`

Ejemplo:

```
1 int a; // Declarar una variable
2 a = 5; // Asignar un valor a una variable
3 int b = 3; // Declarar y asignar a la vez
4 Console.WriteLine(a*b);
```

No se puede cambiar el tipo de una variable!

```
1 int a = 3;
2 a = "hola"; // ERROR!
```


Elementos Básicos

A diferencia de Python, C# es un lenguaje *fuertemente tipado*. Es decir, toda variable debe ser *declarada* con su *tipo* antes de ser usada:

- `<tipo> <nombre variable>;`
- `<tipo> <nombre variable> = <valor>;`

Ejemplo:

```
1 int a; // Declarar una variable
2 a = 5; // Asignar un valor a una variable
3 int b = 3; // Declarar y asignar a la vez
4 Console.WriteLine(a*b);
```

No se puede cambiar el tipo de una variable!

```
1 int a = 3;
2 string a = "hola"; // ERROR!
```

Elementos Básicos

Las variables viven dentro del scope donde se declaran y sus scopes inferiores.

Elementos Básicos

Las variables viven dentro del scope donde se declaran y sus scopes inferiores.

```
1  if (...)  
2  {  
3      int a = 0;  
4      if (...)  
5      {  
6          // "a" es visible desde acá  
7          if (...)  
8          {  
9              // "a" es visible desde acá  
10         }  
11     }  
12     // "a" es visible desde acá  
13 }  
14 // "a" NO EXISTE ACÁ!
```

... pero **no existen** fuera de ellos!

Elementos Básicos

Las variables viven dentro del scope donde se declaran y sus scopes inferiores.

Ejemplo:

```
1 if (true)
2 {
3     int a = 4;
4 }
5 Console.WriteLine(a); // ERROR!!
```

Elementos Básicos

Las variables viven dentro del scope donde se declaran y sus scopes inferiores.

Ejemplo:

```
1 if (true)
2 {
3     int a = 4;
4 }
5 Console.WriteLine(a); // ERROR!!
```

Esto sí funciona:

```
1 int a;
2 if (true)
3 {
4     a = 4;
5 }
6 Console.WriteLine(a); // Esto sí funciona!
```

Tipos de Datos Predefinidos

Tipos de datos predefinidos

Nombre	Bits	Rango	
short	16	-32768	32767
int	32	-2147483648	2147483647
long	64	-9223372036854775808	9223372036854775807
byte	8	0	255
ushort	16	0	65535
uint	32	0	4294967295
ulong	64	0	18446744073709551615
float	32	$\pm 1.5 \times 10^{-45}$	$\pm 3.4 \times 10^{38}$
double	64	$\pm 5.0 \times 10^{-324}$	$\pm 1.7 \times 10^{308}$
decimal	128	$\pm 1.0 \times 10^{-28}$	$\pm 7.9 \times 10^{28}$
char	16	'a', 'b', 'c', ...	
bool	1	true, false	

(float tiene una precisión de 7 dígitos; double de 15 ~ 16 y decimal de 28 ~ 29.)

Tipos de datos predefinidos

Ejemplo:

```
1 short shortVar = 5;
2 long longVar = -10;
3 uint uintVar = 10000;
4 double doubleVar = 0.42e2; // 42 (por defecto se asume que es double)
5 float floatVar = 134.45e-2f; // 1.3445 (la f indica que es float)
6 decimal decimalVar = 1.5e6m; // 1500000 (la m indica que es decimal)
7 char charVar = '&';
8 bool boolVar = false;
```


Tipos de datos predefinidos

Operaciones:

- Números: +, -, *, /, %.
 - Operador // no existe. Pero una división entre variables enteras resulta en //.
 - Operador ** no existe. Hay que usar `Math.Pow(a,b)` – que retorna un `double`.
- Comparaciones: ==, !=, >, >=, <, <=.
- Booleanos: && es el and, || es el or y ! es el not.

Tipos de datos predefinidos

Operaciones:

- Números: +, -, *, /, %.
 - Operador // no existe. Pero una división entre variables enteras resulta en //.
 - Operador ** no existe. Hay que usar `Math.Pow(a,b)` – que retorna un `double`.
- Comparaciones: ==, !=, >, >=, <, <=.
- Booleanos: && es el and, || es el or y ! es el not.

Ejemplo:

```
1 int a = 19;
2 int b = 4;
3 double c = 4;
4 Console.WriteLine(a/b); // 4
5 Console.WriteLine(a/c); // 4.75
6 Console.WriteLine(a < b || b == c); // True
7 Console.WriteLine(a < b && b == c); // False
8 Console.WriteLine(!(a < b && b == c)); // True
```

Tipos de datos predefinidos

Texto:

- `char`: es un solo caracter. Se define con **comillas simples**.
- `string`: es una cadena inmutable de caracteres. Se define con **comillas dobles**.

Tipos de datos predefinidos

Texto:

- char: es un solo caracter. Se define con **comillas simples**.
- string: es una cadena inmutable de caracteres. Se define con **comillas dobles**.

Ejemplo:

```
1 string s = "hola";
2 s += " mundo!"; // "hola mundo!"
3 int N = s.Length; // N = 11 (el largo del string)
4 s = s.Replace("hola","chao"); // s = "chao mundo!"
5 Console.WriteLine(s[0..4]); // s[i..j] es como s[i:j] de python
6 Console.WriteLine(s[^2]); // s[^i] es como s[-i] de python
7 // $"..." es como un f-string de python.
8 int n = 32;
9 Console.WriteLine($"¿Es {n} par? {n%2==0}"); // ¿Es 32 par? True
```

Tipos de datos predefinidos

Arreglos:

- Son “*como*” una lista de largo fijo (definido al momento de ser creadas).
- Todos los elementos deben ser **del mismo tipo**.

Tipos de datos predefinidos

Arreglos:

- Son “*como*” una lista de largo fijo (definido al momento de ser creadas).
- Todos los elementos deben ser **del mismo tipo**.

Sintaxis:

- `<tipo arreglo>[] <nombre variable>;` (declaración)
- `<nombre variable> = new <tipo arreglo>[<largo arreglo>];` (creación)

Tipos de datos predefinidos

Arreglos:

- Son “*como*” una lista de largo fijo (definido al momento de ser creadas).
- Todos los elementos deben ser **del mismo tipo**.

Sintaxis:

- `<tipo arreglo>[] <nombre variable>;` (declaración)
- `<nombre variable> = new <tipo arreglo>[<largo arreglo>];` (creación)

Ejemplo:

```
1 int numElementos = 105;
2 int[] arr = new int[numElementos]; // Creamos un arreglo de ints
3 Console.WriteLine(arr.Length); // Muestra '105' (el largo de arr)
4 Console.WriteLine(arr[17]); // Muestra '0' (valor por defecto int)
5 arr[17] = 4; // Cambiamos el valor en la posición 17 a 4
6 int[] arr2 = arr[16..19]; // [0,4,0]
7 Console.WriteLine(arr2[1]); // Muestra '4'
```

Tipos de datos predefinidos

Hace poco se agregó un tipo de dato *comodín* a C#.

Tipos de datos predefinidos

Hace poco se agregó un tipo de dato *comodín* a C#.

Motivación: A veces definir el tipo de una variable se vuelve redundante.

```
1 int i = 4;  
2 double d = 4.0;  
3 bool b = true;  
4 int[,] arr = new int[10,20];
```

Tipos de datos predefinidos

Hace poco se agregó un tipo de dato *comodín* a C#.

Motivación: A veces definir el tipo de una variable se vuelve redundante.

```
1 int i = 4;  
2 double d = 4.0;  
3 bool b = true;  
4 int[,] arr = new int[10,20];
```

Así que se agregó el tipo `var` que, al momento de compilar el programa, se transforma en el tipo de dato que corresponda a la inicialización de la variable.

```
1 var i = 4;  
2 var d = 4.0;  
3 var b = true;  
4 var arr = new int[10,20];  
5 i = "hola"; // Error! "var" no es "duck typing"
```

Tipos de datos predefinidos

Hace poco se agregó un tipo de dato *comodín* a C#.

Motivación: A veces definir el tipo de una variable se vuelve redundante.

```
1 int i = 4;  
2 double d = 4.0;  
3 bool b = true;  
4 int[,] arr = new int[10,20];
```

Así que se agregó el tipo `var` que, al momento de compilar el programa, se transforma en el tipo de dato que corresponda a la inicialización de la variable.

```
1 var i = 4;  
2 var d = 4.0;  
3 var b = true;  
4 var arr = new int[10,20];  
5 i = "hola"; // Error! "var" no es "duck typing"
```

`var` no provee “*dynamic typing*,” simplemente permite escribir códigos más cortos!

Control de Flujo

Control de flujo

Tenemos if, else if y else.

```
1  int a = 10;
2  if (a < 0)
3  {
4      Console.WriteLine("a es negativo");
5  }
6  else if (a > 0)
7  {
8      Console.WriteLine("a es positivo");
9  }
10 else
11 {
12     Console.WriteLine("a es cero");
13 }
```

Control de flujo

Tenemos if, else if y else.

```
1 int a = 10;
2 if (a < 0)
3     Console.WriteLine("a es negativo");
4 else if (a > 0)
5     Console.WriteLine("a es positivo");
6 else
7     Console.WriteLine("a es cero");
```

Si el statement del if tiene **una** línea se pueden omitir los brackets.

Control de flujo

También tenemos while y for.

```
1 int n = 18;
2 int i = 2;
3 bool isPrime = n > 1;
4 while (i < n && isPrime)
5 {
6     if (n % i == 0)
7         isPrime = false;
8     i++;
9 }
10 Console.WriteLine($"¿Es {n} primo? {isPrime}");
```

Control de flujo

También tenemos while y for.

```
1  int n = 18;
2  int i = 2;
3  bool isPrime = n > 1;
4  while (i < n)
5  {
6      if (n % i == 0)
7      {
8          isPrime = false;
9          break;
10     }
11     i++;
12 }
13 Console.WriteLine($"¿Es {n} primo? {isPrime}");
```


Control de flujo

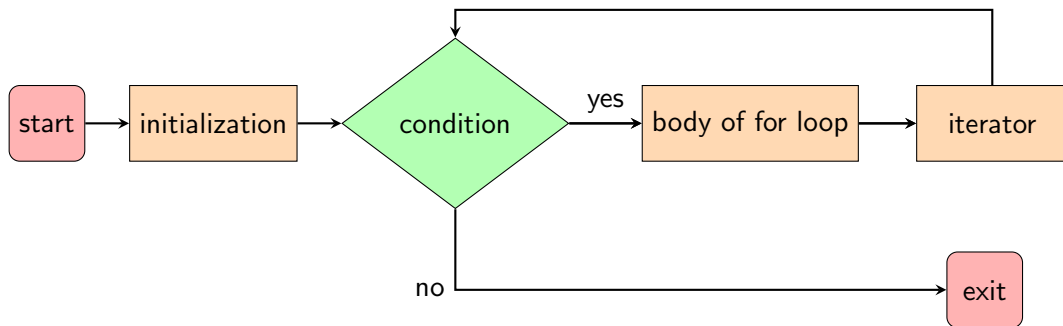
El for **no** funciona como en Python, funciona como en C.

```
1 for (initialization; condition; iterator)
2 {
3     // body of for loop
4 }
```

Control de flujo

El for **no** funciona como en Python, funciona como en C.

```
1 for (initialization; condition; iterator)
2 {
3     // body of for loop
4 }
```



Ejemplo: Mostrar los números del 0 al 9.

```
1 for (int i = 0; i < 10; i++)  
2     Console.WriteLine(i);
```

Ejemplo: Mostrar los números del 0 al 9.

```
1 for (int i = 0; i < 10; i++)  
2     Console.WriteLine(i);
```

Ejemplo: Verificar si un número es primo.

```
1 int n = 18;  
2 bool isPrime = n > 1;  
3 for (int i = 2; i < n && isPrime; i++)  
4     if (n % i == 0)  
5         isPrime = false;  
6 Console.WriteLine($"¿Es {n} primo? {isPrime}");
```

Funciones

(locales)

Sintaxis (parecida a Python):

- **Firma:** `<tipo retorno> <nombre función> (<lista de parámetros>).`
- **Llamado:** `<nombre función> (<lista de inputs>).`

Funciones locales

Sintaxis (parecida a Python):

- **Firma:** <tipo retorno> <nombre función> (<lista de parámetros>).
- **Llamado:** <nombre función> (<lista de inputs>).

```
1  int n = 18;
2  Console.WriteLine($"¿Es {n} primo? {IsPrime(n)}");
3
4  bool IsPrime(int n)
5  {
6      for (int i = 2; i < n; i++)
7      {
8          if (n % i == 0)
9              return false;
10     }
11     return n > 1;
12 }
```

... pero hay que definir el tipo de los parámetros y retorno.

Funciones locales

Usamos void cuando la función no retorna nada.

```
1 ShowWhetherNumIsPrime(18);
2
3 void ShowWhetherNumIsPrime(int n)
4 {
5     bool isPrime = IsPrime(n);
6     Console.WriteLine($"¿Es {n} primo? {isPrime}");
7 }
8
9 bool IsPrime(int n)
10 {
11     for (int i = 2; i < n; i++)
12     {
13         if (n % i == 0)
14             return false;
15     }
16     return n > 1;
17 }
```


Casteo

Sintaxis: Casteo entre tipos compatibles

```
tipo1 var1;  
...  
tipo2 var2 = (tipo2)var1;
```

Sintaxis: Casteo entre tipos compatibles

```
tipo1 var1;  
...  
tipo2 var2 = (tipo2)var1;
```

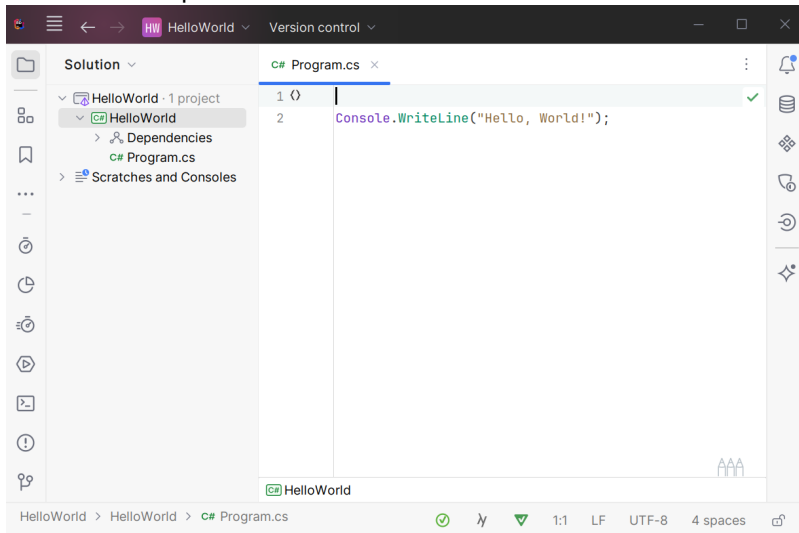
```
1 int i = 26;  
2 uint number = (uint)i;  
3 // Check if number is even  
4 if (number % 2 == 0)  
5     Console.WriteLine(true);  
6 else Console.WriteLine(false);
```

¡Esto sí funciona!

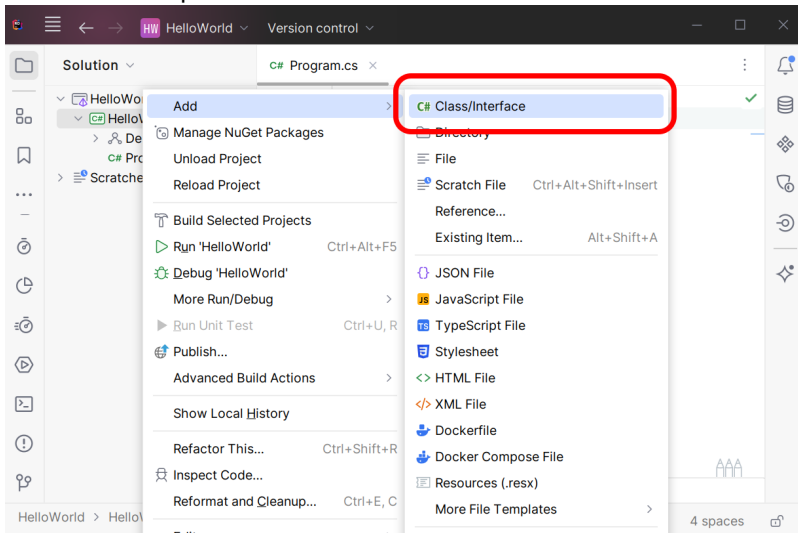
Programación Orientada a Objetos

Así podemos crear una nueva clase en Rider.

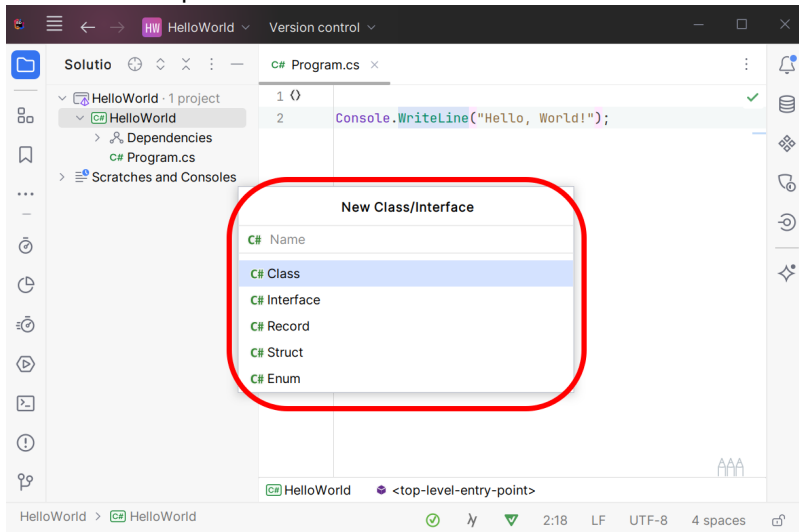
Así podemos crear una nueva clase en Rider.



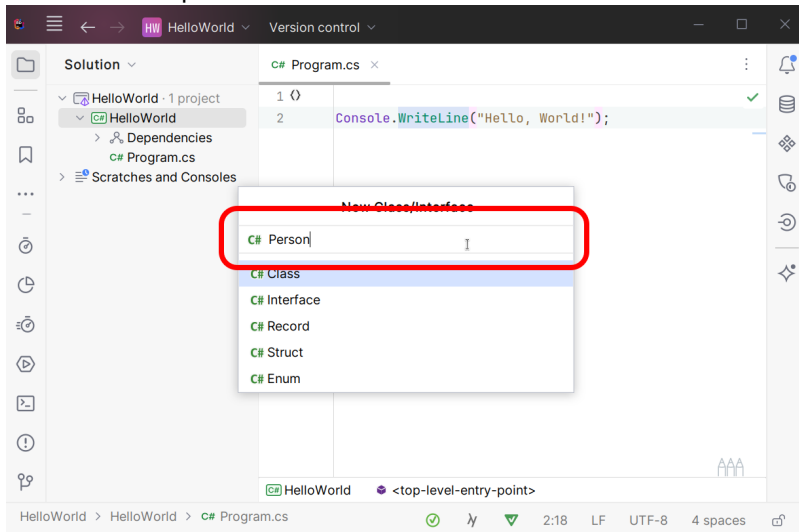
Así podemos crear una nueva clase en Rider.



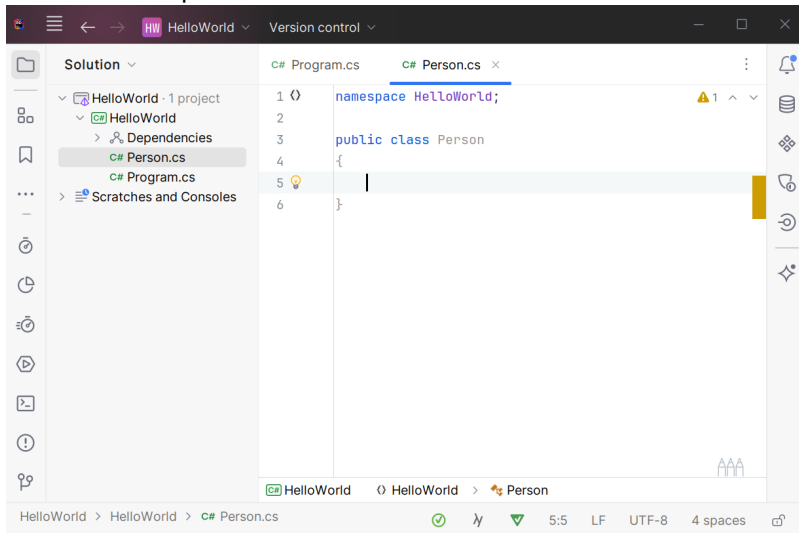
Así podemos crear una nueva clase en Rider.



Así podemos crear una nueva clase en Rider.



Así podemos crear una nueva clase en Rider.



Este es el código que se autogeneró en `Person.cs`

```
2 namespace HelloWorld;  
3  
4 public class Person  
5 {  
6  
7 }
```

¹Por defecto es `internal`.

Este es el código que se autogeneró en `Person.cs`

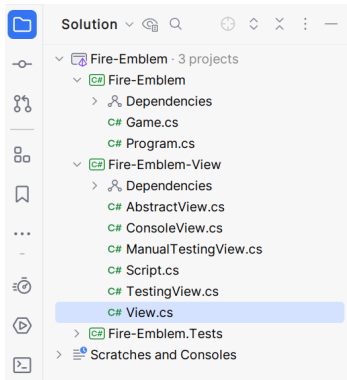
```
2 namespace HelloWorld;
3
4 public class Person
5 {
6
7 }
```

Sintaxis:

- `<access-modifier> class <nombre> { ... }`
- `<access-modifier>` puedes ser:¹
 - `public`: Visible desde otros *assemblies*.
 - `internal`: Visible solo desde su *assembly*.

¹Por defecto es `internal`.

Solo las clases `public` son visibles desde otros proyectos.



Agreguemos un par de atributos a Person.cs

```
2 namespace HelloWorld;
3
4 public class Person
5 {
6     private string name;
7     private int age = 0;
8 }
```

²Por defecto es private.

Agreguemos un par de atributos a Person.cs

```
2 namespace HelloWorld;
3
4 public class Person
5 {
6     private string name;
7     private int age = 0;
8 }
```

Sintaxis:

- `<access-modifier> <tipo> <nombre> = <inicialización>;`
- `<access-modifier>` puedes ser:²
 - `private`: El atributo es accesible solo por métodos de la misma clase.
 - `public`: El atributo es accesible por otras clases.
 - `protected`: El atributo es accesible por los métodos de la clase y sus clases hijas.

²Por defecto es `private`.

Agreguemos un método.

```
2 namespace HelloWorld;
3
4 public class Person
5 {
6     private string name;
7     private int age = 0;
8
9     public void Grow()
10    {
11        age++;
12    }
13 }
```


Agreguemos un método.

```
2 namespace HelloWorld;
3
4 public class Person
5 {
6     private string name;
7     private int age = 0;
8
9     public void Grow()
10    {
11        age++;
12    }
13 }
```

Sintaxis:

- `<access-modifier> <tipo retorno> <nombre>(<tipo1> <arg1>, ...){ }`
- `<access-modifier>` puedes ser: `private`, `public` o `protected`.

Classes

Los métodos pueden *sobrecargarse*: Mismo nombre pero distintos argumentos.

```
2 namespace HelloWorld;
3
4 public class Person
5 {
6     private string name;
7     private int age = 0;
8
9     public void Grow()
10    {
11        age++;
12    }
13
14    public void Grow(int amount)
15    {
16        age = age + amount;
17    }
18 }
```

Finalmente, agreguemos el constructor (que también se puede sobrecargar).

```
1
2 namespace HelloWorld;
3
4 public class Person
5 {
6     private string name;
7     private int age = 0;
8
9     public Person(string name, int age)
10    {
11        this.name = name;
12        this.age = age;
13    }
14
15    public void Grow() { age++; }
16
17    public void Grow(int amount) { age = age + amount; }
18 }
```

Observación 1: El “this” es parecido al “self” de python.

```
8
9  public Person(string name, int age)
10 {
11     this.name = name;
12     this.age = age;
13 }
```

Pero solo es *necesario* usar “this” cuando existen choques de nombres.

Observación 1: El “this” es parecido al “self” de python.

```
8
9  public Person(string name, int age)
10 {
11     this.name = name;
12     this.age = age;
13 }
```

Pero solo es *necesario* usar “this” cuando existen choques de nombres.

Por ejemplo, en el método Grow() es claro que age se refiere al atributo, por lo que no es necesario poner this.age.

```
9  public void Grow()
10 {
11     age++;
12 }
```

Ante ambigüedad, se asume que nos referimos a la variable local.

Ante ambigüedad, se asume que nos referimos a la variable local.

```
2 public class Person
3 {
4     private string name;
5     private int age = 0;
6
7     public Person()
8     {
9         int age = 5;
10        Console.WriteLine(age); // Muestra "5"
11        age++;                  // Suma 1 a la variable local age
12        Console.WriteLine(age); // Muestra "6"
13
14        Console.WriteLine(this.age); // Muestra "0"
15        this.age += 2;              // Suma 2 al atributo age
16        Console.WriteLine(this.age); // Muestra "2"
17    }
18 }
```

Ante ambigüedad, se asume que nos referimos a la variable local.

```
2 public class Person
3 {
4     private string name;
5     private int age = 0;
6
7     public Person()
8     {
9         //int age = 5;
10        Console.WriteLine(age);           // Muestra "0"
11        age++;                             // Suma 1 al atributo age
12        Console.WriteLine(age);           // Muestra "1"
13
14        Console.WriteLine(this.age);      // Muestra "1"
15        this.age += 2;                     // Suma 2 al atributo age
16        Console.WriteLine(this.age);      // Muestra "3"
17    }
18 }
```


Observación 2: Si el método tiene una sola línea se puede usar `=>` en vez de brackets.

Observación 2: Si el método tiene una sola línea se puede usar => en vez de brackets.

```
1 namespace HelloWorld;
2
3 public class Person
4 {
5     private string name;
6     private int age = 0;
7
8     public Person(string name, int age)
9     {
10         this.name = name;
11         this.age = age;
12     }
13
14     public void Grow() { age++; }
15
16     public void Grow(int amount) { age = age + amount; }
17 }
```

Observación 2: Si el método tiene una sola línea se puede usar `=>` en vez de brackets.

```
1 namespace HelloWorld;
2
3 public class Person
4 {
5     private string name;
6     private int age = 0;
7
8     public Person(string name, int age)
9     {
10         this.name = name;
11         this.age = age;
12     }
13
14     public void Grow() => age++;
15
16     public void Grow(int amount) => age = age + amount;
17 }
```

Objetos

Tenemos nuestra clase Person programada en Person.cs.

```
1
2 namespace HelloWorld;
3
4 public class Person
5 {
6     private string name;
7     private int age = 0;
8
9     public Person(string name, int age)
10    {
11        this.name = name;
12        this.age = age;
13    }
14    public void Grow() { age++; }
15    public void Grow(int amount) { age = age + amount; }
16    public void Print() { Console.WriteLine(name+" is "+age+" y/o"); }
17 }
```

Objetos

Tenemos nuestra clase Person programada en Person.cs.

```
1
2 namespace HelloWorld; // <- el namespace nos indica cómo importarla!
3
4 public class Person
5 {
6     private string name;
7     private int age = 0;
8
9     public Person(string name, int age)
10    {
11        this.name = name;
12        this.age = age;
13    }
14    public void Grow() { age++; }
15    public void Grow(int amount) { age = age + amount; }
16    public void Print() { Console.WriteLine(name+" is "+age+" y/o"); }
17 }
```

Agregamos "using HelloWorld;" al main (en Program.cs) para importar Person.

```
2 using HelloWorld;  
3  
4 Person p = new Person("Guillermo", 60);  
5 p.Grow();  
6 p.Grow(12);  
7 p.Print(); // Muestra "Guillermo is 73 y/o"
```

Objetos

Agregamos "using HelloWorld;" al main (en Program.cs) para importar Person.

```
2 using HelloWorld;
3
4 Person p = new Person("Guillermo", 60);
5 p.Grow();
6 p.Grow(12);
7 p.Print(); // Muestra "Guillermo is 73 y/o"
```

Para crear un objeto usamos new:

■ <nombre clase> <nombre variable> = new <nombre clase>(<args>);

Objetos

Agregamos “using HelloWorld;” al main (en Program.cs) para importar Person.

```
2 using HelloWorld;
3
4 Person p = new Person("Guillermo", 60);
5 p.Grow();
6 p.Grow(12);
7 p.Print(); // Muestra "Guillermo is 73 y/o"
```

Para crear un objeto usamos new:

■ <nombre clase> <nombre variable> = new <nombre clase>(<args>);

Llamar a métodos y atributos es igual que en Python. La única diferencia es que si un método/atributo es private no hay forma de llamarlo desde fuera de la clase :)

Namespaces

Namespaces

Hasta ahora nuestro programa usa un namespace llamado HelloWorld.

```
1
2 namespace HelloWorld; // <- el namespace nos indica cómo importarla!
3
4 public class Person
5 {
6     private string name;
7     private int age = 0;
8
9     public Person(string name, int age)
10    {
11        this.name = name;
12        this.age = age;
13    }
14    public void Grow() { age++; }
15    public void Grow(int amount) { age = age + amount; }
16    public void Print() { Console.WriteLine(name+" is "+age+" y/o"); }
17 }
```

Namespaces

¿Qué es el namespace?

- Todo tipo de dato (e.g., clases) se definen dentro de un namespace.
- Los namespaces evitan colisiones de nombres y definen la visibilidad de los tipos que definimos.

Namespaces

¿Qué es el namespace?

- Todo tipo de dato (e.g., clases) se definen dentro de un namespace.
- Los namespaces evitan colisiones de nombres y definen la visibilidad de los tipos que definimos.

Ejemplo:

- Clase `Console` se encuentra definida en el namespace `System`.
- El `using System` se agrega por defecto al código del `main`.
- Técnicamente, para usar `Console` uno debería hacer `System.Console`. Es decir, indicar el namespace y el tipo de dato a usar. Pero si no hay ambigüedades, `C#` deja usar el tipo de dato directo (`Console` en este caso).

Namespaces

¿Qué es el namespace?

- Todo tipo de dato (e.g., clases) se definen dentro de un namespace.
- Los namespaces evitan colisiones de nombres y definen la visibilidad de los tipos que definimos.

Ejemplo:

- Clase `Console` se encuentra definida en el namespace `System`.
- El `using System` se agrega por defecto al código del `main`.
- Técnicamente, para usar `Console` uno debería hacer `System.Console`. Es decir, indicar el namespace y el tipo de dato a usar. Pero si no hay ambigüedades, `C#` deja usar el tipo de dato directo (`Console` en este caso).

Importante: Todos los tipos de datos definidos en un namespace son visibles desde cualquier archivo definido en tu proyecto con el mismo namespace.

Actividad